CS34020/CSM4220 Workshop 4

Motion

Theme: Smart Homes

Purpose and theme: in this workshop focus on motion and what additional information we can get from considering video (i.e. multiple images in a time series). The tasks in this workshop are all around the theme of *smart home environments* but could be applied in other domains.

Part 1: Background Subtraction

In the previous worksheets you have seen how to connect to a video source (file or webcam) and loop over frames to process them. You've also seen how to do image arithmetic such as subtraction, and thresholding, which is a simple way to find foreground objects (when the camera is stationary). The Mixture of Gaussians (MoG) model that we saw in the lectures offers an improved background model, by fitting multiple Gaussians to the values seen at each pixel over a specified number of frames. The largest cluster is usually considered to be the background, and the mean value used in the background subtraction. This allows the background to be continuously updated (e.g. to account for different weather or time of day) but not get corrupted by the passing objects.

Open mog.py and have a look at the code. Most of it is just there to open the video stream and display the different stages. The three lines of code which do the MoG are:

```
fgbg = cv.createBackgroundSubtractorMOG2()
...
fgmask = fgbg.apply(frame)
bg = fgbg.getBackgroundImage()
```

That was easy!

Try running the code and seeing what happens. Read the OpenCV tutorial on Background Subtraction for more information on what it's doing:
https://docs.opencv.org/3.4/d1/dc5/tutorial_background_subtraction.html

An alternative method to MoG which we haven't looked at in the lectures is K nearest neighbour (KNN); you can try this alternative by just switching your background subtraction function to:

```
fgbg = cv.createBackgroundSubtractorKNN()
```

KNN uses a similar approach to MoG, but a different model to cluster the background and foreground colours at each pixel. Both MoG and KNN have a number of optional parameters you can play with; have a look at the documentation and try playing with them:
https://docs.opencv.org/3.4/de/de1/group__video__motion.html#ga2beb2dee7a073809ccec60f145b6b29c

Part 2: Optical Flow

We will put both sparse and dense optical flow into action here. Both are explained in detail in this tutorial, and this is the basis for the code I have supplied:
https://docs.opencv.org/3.4/d4/dee/tutorial_optical_flow.html

Sparse Optical Flow (Lucas-Kanade)

Open the file sparse-optical-flow.py. The implementation of sparse optical flow in OpenCV is the Lucas-Kanade method that we looked at in the lecture. Again, a lot of the code in this file is running the video file and drawing the results, they key lines at the start are:

```
# params for ShiTomasi corner detection
feature_params = dict(maxCorners = 100, qualityLevel = 0.3,
                      minDistance = 7, blockSize = 7 )

# Parameters for lucas kanade optical flow
lk_params = dict(winSize  = (15, 15), maxLevel = 2, criteria=
           (cv.TERM_CRITERIA_EPS|cv.TERM_CRITERIA_COUNT,10,0.03))
# Take first frame and find corners in it ret,
old_frame = cap.read()
old_gray = cv.cvtColor(old_frame, cv.COLOR_BGR2GRAY)
p0 = cv.goodFeaturesToTrack(old_gray, mask=None,**feature_params)
```

and the key lines in the loop are:

```
p1,st,err = cv.calcOpticalFlowPyrLK( old_gray, frame_gray, p0,
                                     None, **lk_params)
# Select good points
if p1 is not None:
     good_new = p1[st==1]
     good_old = p0[st==1]
```

You can see the main steps which we outlined in the lecture:

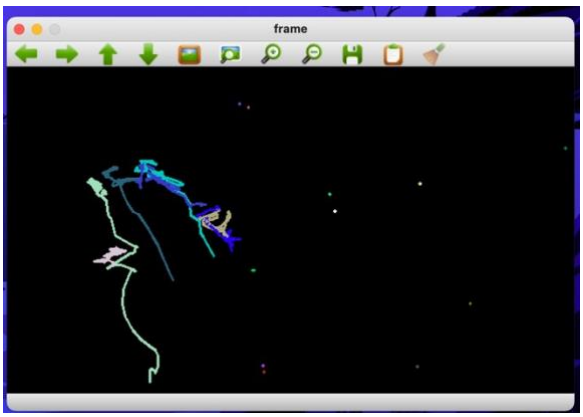
1. Identifying some features to track;
2. Using a window (neighbourhood) to follow features;
3. And only keeping points that give useful results.

There's one thing to note about this code if you are not used to Python: `feature_params` and `lk_params` are both 'dictionaries', which means they are a collection of named variables, a bit like a struct in other languages. Where the dictionaries are named in a function, e.g.

```
p0 = cv.goodFeaturesToTrack(old_gray, mask=None,**feature_params)
```

it works the same as any other variable – it takes the contents of `feature_params` and puts it into the function, including the parameter names.

You can see how this is a chain of tools, and you can modify any parts within this. You might remember from a previous workshop that `cv.goodFeaturesToTrack` is KLT (ShiTomasi) corners, so you can exchange this for Harris corners (`cv.cornerHarris`) and see the difference.



There are also a lot of parameters to tweak in the Lucas-Kanade implementation, including window size for the neighbourhood, have a look at the OpenCV Docs[1] and try modifying some of the parameters.

See if you can modify the code to only show the optical flow, without the picture, e.g.:

Dense Optical Flow (Farneback)

Open the file dense-optical-flow.py. The implementation of dense optical flow in OpenCV is the Farneback method, which we saw in the lecture. Again, a lot of the code in this file is running the video file and drawing the results, they key line here is:

```
# calculate the optical flow
flow = cv.calcOpticalFlowFarneback( prvs, next, None, 0.5, 3,
                                    15, 3, 5, 1.2, 0)
```

The parameters for Farneback are similar, and again detailed in the OpenCV Docs[2]. The code comments should help you to understand mapping the vectors onto the HSV colour space. Try modifying the code so that the original frame is displayed in the background behind the optical flow results:



Part 3: Object tracking

Often, we are interested in tracking a box surrounding an object, rather than individual points. OpenCV provides a large number of object bounding box trackers to choose from, with different behaviours and trade-offs between quality and speed. Here we will experiment with a few different trackers. Open the file obj_tracker.py, which uses the MobileNet+SSD object detector that we saw last week to find an object in the frame. Once an object is

[1] https://docs.opencv.org/3.4/dc/d6b/group__video__track.html#ga473e4b886d0bcc6b65831eb88ed93323
[2] https://docs.opencv.org/3.4/dc/d6b/group__video__track.html#ga5d10ebbd59fe09c5f650289ec0ece5af

detected, the detector is “switched off” and the bounding box of the detected object is tracked.

The provided code includes a method to return a tracker from a range of those available:

```
trackerTypes = ['BOOSTING', 'MIL', 'KCF','TLD', 'MEDIANFLOW',
                'MOSSE', 'CSRT']

def createTrackerByName(trackerType):
  # Create a tracker based on tracker name
  if trackerType == trackerTypes[0]:
    tracker = cv.legacy.TrackerBoosting_create()
  elif trackerType == trackerTypes[1]:
    tracker = cv.TrackerMIL_create()
  elif trackerType == trackerTypes[2]:
    tracker = cv.legacy.TrackerKCF_create()
  elif trackerType == trackerTypes[3]:
    tracker = cv.legacy.TrackerTLD_create()
  elif trackerType == trackerTypes[4]:
    tracker = cv.legacy.TrackerMedianFlow_create()
  elif trackerType == trackerTypes[5]:
    tracker = cv.legacy.TrackerMOSSE_create()
  elif trackerType == trackerTypes[6]:
    tracker = cv.TrackerCSRT_create()
  else:
    tracker = None
    print('Incorrect tracker name')
    print('Available trackers are:')
    for t in trackerTypes:
      print(t)

  return tracker
```

You’ll notice that some of the trackers are in the legacy package, but they still work fine. Add the following code after retrieving the output of the network (after the comment “# Initialize the tracker here”):

```
if results.shape[2]>0: # Detected some objects
     # Loop over objects
```

```
    for i in range(results.shape[2]):
        if results[0,0,i,2]>0.5: # confidence > 50%
            # Object detected, get the bounding box
            x1, y1, x2, y2 = results[0, 0, 0, 3:7]
            ih, iw, ic = frame.shape
            x1, x2 = int(x1*iw), int(x2*iw)
            y1, y2 = int(y1 * ih), int(y2 * ih)
            tracking = True # Disables detection
            # Make the tracker
            tracker = createTrackerByName(trackerType)
            # Initialise the tracker
            bbox = [x1,y1,x2-x1,y2-y1]
            tracker.init(frame, bbox)
            break # Stop looking for detections
```

Now the tracker has started frames will be passed to the `else` clause. Put the following code in the else clause (after "# Update the tracking here")

```
# Update tracker with current frame
(success, box) = tracker.update(frame)
if success:
  # Get the bounding box and display it
  (x, y, w, h) = [int(v) for v in box]
   cv.rectangle(frame, (x, y), (x+w, y+h), (203, 192, 255), 2)
```

Try modifying the trackerType variable to see how the different trackers behave. Which do you think work best in this application? Is it consistent across videos?

Part 4: Pose tracking

Tracking the person's location (with a bounding box) can provide some useful information, but tracking the pose of the person (or people) in the scene could provide more and richer information. Here we will set up a lightweight deep learning pose tracker for our hypothetical smart home. The process is very similar to what we did for object detection. Here we will use a lightweight version of the OpenPose model, as described here:

https://arxiv.org/pdf/1811.12004

and the opencv usage is described here:

https://blog.roboflow.com/what-is-openpose/

https://cv-tricks.com/pose-estimation/using-deep-learning-in-opencv/

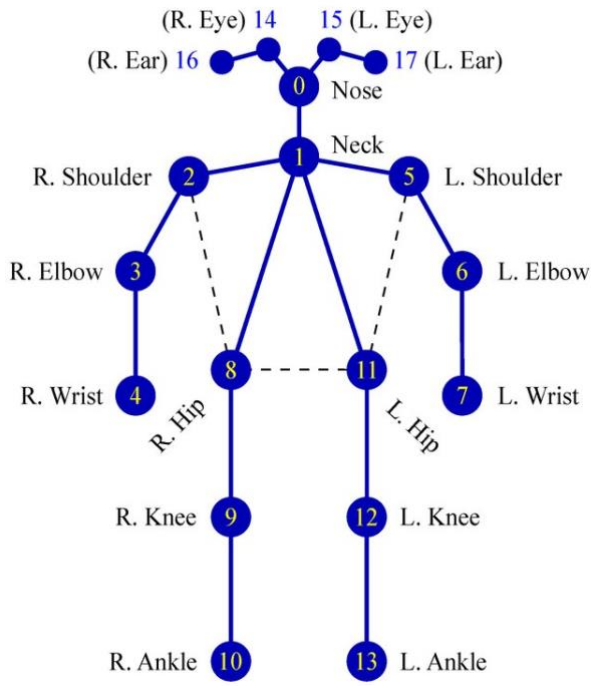
We will use a TensorFlow version of the lightweight model, trained on the COCO dataset, with 18 keypoints. The model can be read from the file graph_opt.pb using:

```
net = cv.dnn.readNetFromTensorflow("graph_opt.pb")
```

The normalisation parameters are similar to the MobileNet+SSD settings:

```
# The required input shape and normalisation for the network
input_shape = (368, 368)
mean = (127.5, 127.5, 127.5)#(0,0,0)
scale = 1.0# 1.0/255.0
nPoints = 18 # Coco 18 point model
threshold = 0.5
```

The points in the model are shown below:



(Figure from Lin, F.-C., Ngo, H.-H., Dow, C.-R., Lam, K.-H., & Le, H. L. (2021). Student Behavior Recognition System for the Classroom Environment Based on Skeleton Pose Estimation and Person Detection. *Sensors*, *21*(16), 5314. https://doi.org/10.3390/s21165314)

For drawing the model, it is useful to store the links in the skeleton model, i.e.:

```
pointPairs = [[16,14],[15,17],[14,0],[15,0],[0,1], # Head
              [2,1],[3,2],[3,4],  # right arm
              [1,5],[5,6],[6,7], # left arm
              [1,8],[1,11], # torso
              [8,9],[9,10], # right leg
              [11,12],[12,13] # left leg
```

```
                ]
```

Then, in the main looop, we need to create the normalised image ("blob") and pass it through the network. We will also store the frame size and the results window size for use later in displaying the pose:

```
frameWidth = frame.shape[1]
frameHeight = frame.shape[0]

blob = cv.dnn.blobFromImage(frame, scalefactor=scale,
                            size=input_shape, mean=mean,
                            swapRB=True, crop=False)
net.setInput(blob)
results = net.forward()

H = results.shape[2]
W = results.shape[3]
```

The results tensor has the format [*image_id, data_id, width, height*] representing a set of 2D probablility maps (of size *width* by *height*), one for each image passed to the network in a batch (indexed by *image_id*) and one for each point plus for each affinity map (we won't be using these here), all packed together and indexed by *data_id*. Here we will just use the first 18 probability maps, which should give a high value for the expected location. The output probability maps are only 46x46 pixels, limiting the accuracy of the detector.

To find the max value in the ith probability map, we can use the minMaxLoc opencv function:

```
probMap = results[0, i, :, :]
# Find global maxima of the probMap.
minVal, prob, minLoc, point = cv.minMaxLoc(probMap)
```

Here the max value is stored in the `prob` variable and the location of the max value is stored in the `point` variable. If the probability value is larger than a threshold, we can rescale the detected point to the correct frame size, draw the point and include it in our list of point detections:

```
if prob > threshold :
      # Scale the point to fit on the original image
      x = (frameWidth * point[0]) / W
      y = (frameHeight * point[1]) / H
      cv.circle(frame, (int(x), int(y)), 5,
                (0, 255, 255), thickness=-1,
```

```
                lineType=cv.FILLED)

        # Add the point to the list
        points.append((int(x), int(y)))
else :
        points.append(None)
```

Finally, we can draw lines when both ends of the line segment have been detected. Try the tracker out, you'll notice that the performance isn't great ☹. Consider ways you might be able to get improved performance if you needed to deploy such a system.

Link to Assignment

In the assignment you are asked to detect yellow cars. Consider the following questions relating to the assignment, and **write down your answers**, you will need them for the assignment report.

Questions for both CS34020 Students

Write a short paragraph discussing which of the methods used in this workshop might be suitable to help with the assignment task and which are not, and why. You will be asked to include this in your assignment submission.

Questions for CSM4220 Students

Find a recent academic paper (last 5 years) that describes another object tracking algorithm. Write a short paragraph discussing comparing with methods used in this workshop, and discuss which might be most suitable to help with the assignment task and which are not, and why. You will be asked to include this in your assignment submission.

Submission

Please **submit your work from this workshop**, to Blackboard at the end of the practical. The submissions themselves are not assessed and carry no marks, but will be used as an additional measure of **engagement**. Don't worry if you haven't completed all aspects of this worksheet, just submit what you have done.